

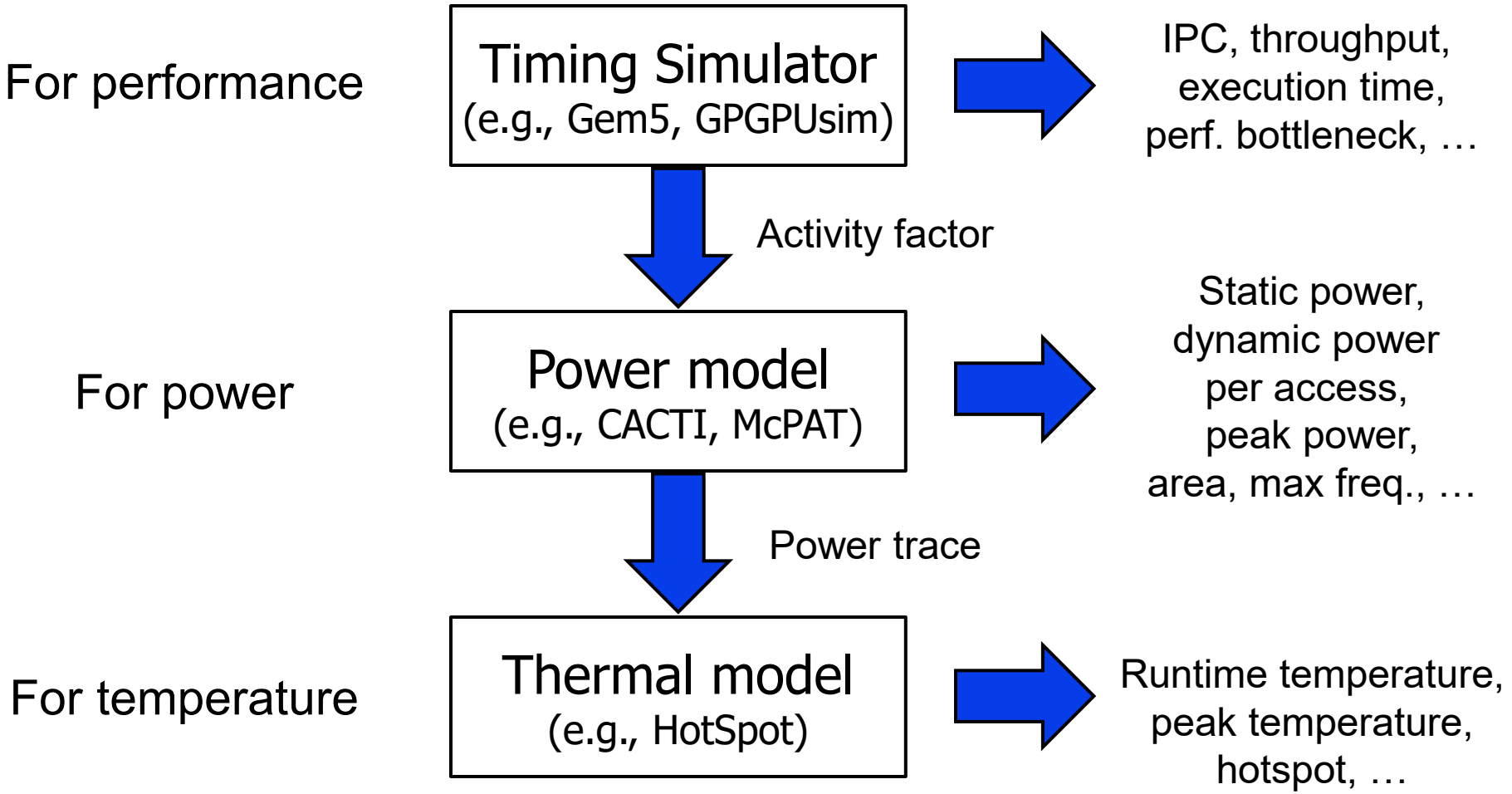

# Project

Dongmoon Min
(Sungkyunkwan University)
dongmoon.min@skku.edu

# Project Overview

- **Goal:** build your own performance–power–temperature evaluation flow, then use it to study a real architecture idea
- **Two problems (25 points)**
  - **Problem 1 (10 pts):** connect gem5 + McPAT + HotSpot and simulate multi-core out-of-order execution
  - **Problem 2 (15 pts):** implement an existing architecture technique, and propose an advanced solution
- **Team:** 2~3 students per team
- **Deliverables:** 팀번호_이름1_이름2_...이름N.zip
  - Report (≤ 6 pages, pdf) submitted on iCampus
  - Codes
    - Problem #1: Glue code
    - Problem #2: modified code in diff files
- **Due:** The end of this semester (~12/18 23:59)

# Problem 1: What to build



# Problem 1: Experiments

**Parameter sweeps on a multi-core out-of-order CPU**

| Sweep | Configurations | Fixed | Question |
|---|---|---|---|
| A. Core count | **1, 4, 8** cores | issue width (e.g., 4-wide) | How does thread-level parallelism pay off in speedup, power, and temperature? |
| B. Superscalar width | **2, 4, 8**-wide (fetch / issue / commit) | core count (e.g., 4 cores) | How does instruction-level parallelism pay off — and what does it cost? |

- **Workloads:** PARSEC 3.0, simsmall input — blackscholes, canneal, freqmine, streamcluster (ROI only)
- **Metrics (per configuration, per workload)**
  - **Normalized speedup** — relative to the smallest configuration of each sweep
  - **Average power (W)** — dynamic + static, whole chip
  - **Steady-state temperature (°C)**

# Problem 1: What to Analyze

- **Speedup:** does it scale with cores? with width?
- **Power:** where does the power go (core vs. cache vs. interconnect, dynamic vs. static)? Does power grow linearly with cores? With width?
- **Temperature:** How does the temperature change with cores and width?

- **Grading (10 pts)**
  - All configurations with correct metrics and plots (5pts)
  - Quality of analysis (5pts)

  - No code ➔ 0pt!

# Problem 2: Improve a Real Technique

- **Step 1 — Pick an architecture technique** you are interested in
  - e.g., TAGE / perceptron predictor, stride or stream prefetcher, cache replacement (RRIP), issue-queue / wakeup optimization, memory-dependence prediction, DVFS or power-gating policy, …
- **Step 2 — Highlight its limitation and propose a new solution**
- **Step 3 — Implement them in gem5**
- **Step 4 — Evaluate your solution with your Problem-1 flow**: performance, power, and temperature

- **Rule:** baseline, original technique, and your improvement must be compared under the same configuration and workloads

# Problem 2: Evaluation and Grading

- **Describe your idea clearly!!**
  - With limitation of the previous work
- **Compare your idea while clarifying the tradeoffs**
  - Performance gain vs. power overhead vs. temperature
    - e.g., a 10% IPC gain with 5% higher power consumption
- **Suggested figures**
  - Speedup, average power, and peak temperature per workload for {baseline, technique, your improvement}
- **Grading (15 pts)**
  - Novelty and plausibility of your solution (10pts)
  - Analysis of the results (5pts)
  - No code and result ➔ 0pt!

# Report and Submission

- **Report: at most 6 pages** including figures and tables; PDF
- **Suggested structure**
  - 1. Problem 1 — How you build the tool, results & analysis
  - 2. Problem 2 — Previous idea & limitation, proposal, results & analysis
  - 3. Conclusion — what you learned
    - how you used AI, what you would do with more time, …
- **AI policy – Write the report yourselves!!**
  - Use AI tools for coding; mention briefly how you used them in Conclusion
- **Submit on iCampus:** zip file
  - Zip (name: "팀번호_이름1_...이름N.zip")
    - report (pdf)
    - problem1 (folder)
      - Codes, … (mainly glue codes)
    - problem2 (folder)
      - Codes, … (mainly diff files)
- **Due:** Until the end of this semester (~12/18 23:59)

# Good Luck!

- If you have any questions, upload them in Q&A board
  - I/TAs will not response any personal email about the project